Flutter Layout Widgets Demo

A simple explanation of the widgets and properties used in the project

This guide explains the project in beginner-friendly words. Use it alongside the Dart files and try changing the values to see what happens.

1. Project Structure

The app has a main.dart file and a separate Dart file for each example screen. Keeping screens separate makes the code easier to read and maintain.

```
lib/
  main.dart
  screens/
   container_screen.dart
   row_column_screen.dart
   nested_layout_screen.dart
   buttons_snackbar_screen.dart
   alert_dialog_screen.dart
   padding_screen.dart
   sizedbox_container_screen.dart
```

In main.dart, MaterialApp starts the app. HomeScreen displays a list of examples. When you tap an item, Navigator.push opens its screen.

2. Basic Flutter App Widgets

MaterialApp

Provides the app's Material Design structure, theme, and navigation setup.

Scaffold

A basic page structure. It commonly contains an AppBar, body, and other page areas.

AppBar

The bar at the top of a screen, usually showing its title.

Text

Displays words or sentences.

Icon

Displays a built-in icon, such as an email, person, or home symbol.

ListView.builder

Builds a scrollable list efficiently. itemCount says how many list items are needed; itemBuilder creates each item.

Card and ListTile

Card provides a visual surface; ListTile is a convenient row with a title and optional leading/trailing widgets.

Navigator.push

Moves from the home screen to another screen.

MaterialPageRoute

Defines the page transition route used when opening a screen.

3. Container and Its Properties

Container is a flexible box. It can set a child's size, add space around the child, align the child, and decorate the box.

Property	Simple meaning
height	The height of the box.
width	The width of the box.
padding	Space inside the box, between its edge and child.
margin	Space outside the box, separating it from other widgets.
alignment	Where the child sits inside the box, for example Alignment.center.
color	A simple background color. For more decoration, use BoxDecoration.
decoration	A collection of visual settings such as color, border, and rounded corners.
borderRadius	Rounds the corners of the box.
border	Draws an outline around the box.

BoxDecoration

BoxDecoration is used to style the Container. In the examples, it adds a background color, rounded corners using BorderRadius.circular(...), and an outline using Border.all(...).

4. Row and Column

Row places its children next to each other horizontally (left to right). **Column** places children vertically (top to bottom).

Main axis and cross axis

For a Row, the main axis is horizontal and the cross axis is vertical. For a Column, the main axis is vertical and the cross axis is horizontal.

mainAxisAlignment

Controls how children are distributed along the main direction. Example: spaceEvenly puts even space around each child.

crossAxisAlignment

Controls alignment along the other direction. Example: center centers children across the cross axis; start aligns them at the beginning.

children

A list of widgets placed inside a Row or Column.

Expanded

Makes a child use available leftover space in a Row or Column. Often helps prevent text from overflowing.

Flexible

Lets a child fit within the available space without necessarily forcing it to fill all of it.

SizedBox

Can create a gap between widgets, such as SizedBox(height: 12) between vertically stacked items.

5. Nested Rows and Columns

Nested means one layout widget is placed inside another. For example, a Column can contain cards, and each card can contain a Row with an icon and a Column with a title and subtitle.

A sample structure from the screen is:

```
Column
  ■■ Container (information tile)
     ■■ Row
       ■■ Icon
       ■■ Expanded
         ■■ Column
           ■■ Text (title)
           ■■ Text (subtitle)
```

Expanded gives the text area the remaining row width. This is useful when text length can change.

6. Buttons and SnackBar

ElevatedButton

A button with an elevated visual style.

FilledButton

A filled button with a strong background.

OutlinedButton

A button with an outline.

TextButton

A simple text-style button.

IconButton

A button that displays an icon.

onPressed

The callback that runs when the button is tapped. If it is null, the button is disabled.

ScaffoldMessenger.of(context)

Finds the message controller for the current page's Scaffold.

showSnackBar

Shows a short message at the bottom of the screen.

SnackBar duration

How long the message stays visible, for example Duration(seconds: 2).

SnackBarBehavior.floating

Makes the SnackBar appear floating above the bottom edge instead of attached to it.

SnackBarAction

Adds an action button, such as UNDO, to the SnackBar.

7. AlertDialog

AlertDialog is a popup used to show a message or ask the user to confirm an action.

showDialog

Opens the popup dialog.

AlertDialog title

The heading at the top of the dialog.

content

The message or details shown inside the dialog.

actions

Buttons displayed at the bottom, such as Cancel and Delete.

Navigator.pop(dialogContext, value)

Closes the dialog and can return a result. In this example, true means confirmed and false means cancelled.

barrierDismissible

If false, tapping outside the popup does not close it.

async / await

Allows the screen to wait for the user's dialog choice before continuing.

context.mounted

Checks that the screen is still active before using its context after an async operation.

8. Padding and Margin

Padding is inner spacing: it creates room between the widget's edge and its child. Margin is outer spacing: it creates room around the widget, separating it from nearby widgets.

EdgeInsets.all(value)

Applies the same padding to all four sides.

EdgeInsets.only(left, top, right, bottom)

Sets each side separately.

EdgeInsets.symmetric(horizontal, vertical)

Sets left/right with horizontal and top/bottom with vertical.

The project uses Padding as a widget and also uses Container's padding property. Both are ways to create inner space.

9. SizedBox vs Container

SizedBox is mainly used to give a widget a size or create empty space. Container can also set size, but offers more features such as padding, alignment, decoration, and margin.

SizedBox	Container
Good for fixed or calculated width/height.	Can set width and height too.
Commonly used for gaps: SizedBox(height: 12).	Can create a gap, but SizedBox is usually clearer for this.

SizedBox	Container
Can wrap a child.	Can wrap a child and style its box.
Does not provide BoxDecoration.	Supports color, border, rounded corners, etc. through decoration.

10. Responsive Layout and Relative Dimensions

A fixed width like 200 pixels may look fine on one device but too small or too large on another. This project uses LayoutBuilder to read the available space.

LayoutBuilder

LayoutBuilder provides constraints, including constraints.maxWidth and constraints.maxHeight. These tell you how much room the parent has made available.

```
final boxWidth = constraints.maxWidth * 0.90;
Container(width: boxWidth)
```

Here, 0.90 means 90% of the available width. The same approach is used for example heights based on the available width.

Other responsive helpers

Expanded and Flexible share space inside Row or Column. double.infinity asks a widget to use all the width allowed by its parent. Remember: a widget cannot exceed the constraints its parent gives it.

> Responsive sizing is not only about replacing every number. Small spacing values (such as 8, 12, or 16) are often intentionally fixed so margins and padding remain consistent.

11. Other Properties and Helpers Used

SingleChildScrollView

Makes content scroll if it is taller than the screen.

LayoutBuilder

Lets the screen adapt its layout using available constraints.

BoxConstraints / constraints.maxWidth

The available size information supplied to the widget.

mainAxisSize: MainAxisSize.min

Makes a Column take only the height needed by its children.

crossAxisAlignment: CrossAxisAlignment.stretch

Stretches children across the available cross-axis width in a Column.

const

Marks values/widgets that do not change, helping Flutter reuse them efficiently.

StatelessWidget

A widget whose displayed structure does not manage its own changing state.

BuildContext

A reference to a widget's position in the widget tree; used by functions like Navigator and ScaffoldMessenger.

12. Quick Practice Ideas

- Change ContainerScreen's borderRadius and border color.
- Change Row's mainAxisAlignment from spaceEvenly to spaceBetween or center.
- Try crossAxisAlignment.start and crossAxisAlignment.end in a Column.
- Remove Expanded from a nested row and observe how long text behaves.
- Change the SnackBar duration and behavior.
- Change AlertDialog's title, content, and action labels.
- Compare EdgeInsets.all with EdgeInsets.symmetric and EdgeInsets.only.
- Resize the app window or run on another device to see relative sizing adapt.

Key idea: Row, Column, and GridView arrange widgets. Container and Padding help size and style them. SizedBox is useful for sizing and spacing. Responsive layouts use the space offered by the parent instead of assuming every screen is the same size.